

datasource_transformations [↗](#)

Module that handles datasource/feature related transformations/conversion.

DEFAULT_SPARK_APP_NAME module-attribute [↗](#)

```
DEFAULT_SPARK_APP_NAME = 'pySparkJob-created-by-ds_api'
```

FINAL_TIMESTAMP module-attribute [↗](#)

```
FINAL_TIMESTAMP = datetime(2200, 1, 1)
```

INITIAL_TIMESTAMP module-attribute [↗](#)

```
INITIAL_TIMESTAMP = datetime(1970, 1, 1)
```

TagUtils module-attribute [↗](#)

```
TagUtils = autoclass(_AUTOML_TRANSFORMATIONS_CLASSPATH + 'TagUtils')
```

logger module-attribute [↗](#)

```
logger = getLogger(__name__)
```

CompositeField dataclass [↗](#)

```
CompositeField(desc: str, tags: Sequence[str], fields: Sequence[str])
```

Bases: [SemanticField](#)

Defines a composite field to use in the semantics construction.

Please check the `available_tags` property to get the list of supported tags.

TAGS_MAP class-attribute instance-attribute [↗](#)

```
TAGS_MAP = {'address': ADDRESS_TAG, 'line1': LINE1_TAG, 'line2': LINE2_TAG, 'zip': ZIP_TAG, 'city': CITY_TAG, 'region': REGION_TAG, 'country': COUNTRY_TAG, 'coordinate': COORDINATE_TAG, 'longitude': LONGITUDE_TAG, 'latitude': LATITUDE_TAG, 'location': LOCATION_TAG, 'datetime': DATETIME_TAG, 'expiration-date': EXPIRATION_DATETIME_TAG, 'yymm-datetime': YYMM_DATETIME_TAG, 'name': NAME_TAG, 'email': EMAIL_TAG, 'amount': AMOUNT_TAG, 'available-amount': AVAILABLE_AMOUNT_TAG, 'entity': ENTITY_TAG, 'grouping-entity': GROUPING_ENTITY_TAG, 'acquirer': ACQUIRER_TAG, 'card': CARD_TAG, 'customer': CUSTOMER_TAG, 'ip': IP_TAG, 'merchant': MERCHANT_TAG}
```

available_tags property [↗](#)

```
available_tags: List[str]
```

Returns the list of available tags.

desc instance-attribute [↗](#)

```
desc: str
```

fields instance-attribute [¶](#)

```
fields: Sequence[str]
```

tags instance-attribute [¶](#)

```
tags: Sequence[str]
```

to_pulse [¶](#)

```
to_pulse(tagged_fields: Sequence)
```

Turns our CompositeField into its pulse representation.

Parameters:

Name	Type	Description	Default
<code>tagged_fields</code> ¶	sequence of converted TaggedField	Set of possible simple tagged fields that can be used to compose the composite field.	<i>required</i>

Returns:

Type	Description
A pulse representation of the composite field.	

SemanticField dataclass [¶](#)

```
SemanticField()
```

Superclass for TaggedField and CompositeField

TAGS_MAP class-attribute instance-attribute [¶](#)

```
TAGS_MAP = {'address': ADDRESS_TAG, 'line1': LINE1_TAG, 'line2': LINE2_TAG, 'zip': ZIP_TAG, 'city': CITY_TAG, 'region': REGION_TAG, 'country': COUNTRY_TAG, 'coordinate': COORDINATE_TAG, 'longitude': LONGITUDE_TAG, 'latitude': LATITUDE_TAG, 'location': LOCATION_TAG, 'datetime': DATETIME_TAG, 'expiration-date': EXPIRATION_DATETIME_TAG, 'yymm-datetime': YYMM_DATETIME_TAG, 'name': NAME_TAG, 'email': EMAIL_TAG, 'amount': AMOUNT_TAG, 'available-amount': AVAILABLE_AMOUNT_TAG, 'entity': ENTITY_TAG, 'grouping-entity': GROUPING_ENTITY_TAG, 'acquirer': ACQUIRER_TAG, 'card': CARD_TAG, 'customer': CUSTOMER_TAG, 'ip': IP_TAG, 'merchant': MERCHANT_TAG}
```

available_tags property [¶](#)

```
available_tags: List[str]
```

Returns the list of available tags.

Semantics [¶](#)

Utility class to convert to/from a json semantic file.

from_semantic_file staticmethod [¶](#)

```
from_semantic_file(filename: str, path: Optional[str] = None) -> Tuple[SemanticField, ...]
```

Takes a semantic file and converts it into a usable collection of SemanticField.

Parameters:

Name	Type	Description	Default
<code>filename</code> ¶	<code>str</code>	Filename of the semantic file.	<i>required</i>
<code>path</code> ¶	<code>str</code>	Path where the file is to be saved, default is the working directory	<code>None</code>

Returns:

Type	Description
<code>tuple of SemanticField</code>	A collection of SemanticField.

`to_semantic_file` `staticmethod` [¶](#)

```
to_semantic_file(semantic: Sequence[SemanticField], filename: str = 'semantic_file.json', path: Optional[str] = None)
```

Takes a collection of semantics and writes a json semantic file.

Parameters:

Name	Type	Description	Default
<code>semantic</code> ¶	<code>sequence of SemanticField</code>	Semantics to write.	<i>required</i>
<code>filename</code> ¶	<code>str</code>	Name for the semantic file, default is "semantic_file.json"	<code>'semantic_file.json'</code>
<code>path</code> ¶	<code>str</code>	Path where the file is to be saved, default is the working directory	<code>None</code>

`Split` `dataclass` [¶](#)

```
Split(name: str, start_timestamp: Optional[Union[int, datetime]] = None, end_timestamp: Optional[Union[int, datetime]] = None, path: Optional[str] = None)
```

Defines a datasource split.

Attributes:

Name	Type	Description
<code>name</code>	<code>str</code>	Name of the datasource created by the split
<code>start_time_stamp</code>	<code>(optional, int or datetime)</code>	Timestamp starting the splitting interval. If not specified, the beginning of the dataset will be included and end_timestamp must be specified.
<code>end_timestamp</code>	<code>(optional, int or datetime)</code>	Timestamp ending the splitting interval. If not specified, the end of the dataset will be included and start_timestamp must be specified.
<code>path</code>	<code>(optional, str)</code>	The writing path for the split.

end_timestamp class-attribute instance-attribute [¶](#)

```
end_timestamp: Optional[Union[int, datetime]] = None
```

name instance-attribute [¶](#)

```
name: str
```

path class-attribute instance-attribute [¶](#)

```
path: Optional[str] = None
```

start_timestamp class-attribute instance-attribute [¶](#)

```
start_timestamp: Optional[Union[int, datetime]] = None
```

to_pulse [¶](#)

```
to_pulse()
```

TaggedField dataclass [¶](#)

```
TaggedField(desc: str, tags: Sequence[str])
```

Bases: [SemanticField](#)

Defines a tagged field to use in the semantics construction.

Please check the `available_tags` property to get the list of supported tags.

TAGS_MAP class-attribute instance-attribute [¶](#)

```
TAGS_MAP = {'address': ADDRESS_TAG, 'line1': LINE1_TAG, 'line2': LINE2_TAG, 'zip': ZIP_TAG, 'city': CITY_TAG, 'region': REGION_TAG, 'country': COUNTRY_TAG, 'coordinate': COORDINATE_TAG, 'longitude': LONGITUDE_TAG, 'latitude': LATITUDE_TAG, 'location': LOCATION_TAG, 'datetime': DATETIME_TAG, 'expiration-date': EXPIRATION_DATETIME_TAG, 'yymm-datetime': YYMM_DATETIME_TAG, 'name': NAME_TAG, 'email': EMAIL_TAG, 'amount': AMOUNT_TAG, 'available-amount': AVAILABLE_AMOUNT_TAG, 'entity': ENTITY_TAG, 'grouping-entity': GROUPING_ENTITY_TAG, 'acquirer': ACQUIRER_TAG, 'card': CARD_TAG, 'customer': CUSTOMER_TAG, 'ip': IP_TAG, 'merchant': MERCHANT_TAG}
```

available_tags property [¶](#)

```
available_tags: List[str]
```

Returns the list of available tags.

desc instance-attribute [¶](#)

```
desc: str
```

tags instance-attribute [¶](#)

```
tags: Sequence[str]
```

to_pulse [¶](#)

```
to_pulse()
```